

Índice

1. Fundamentos de Python: bases y orientación a objetos. Parte 1	2
1.1. Características y Paradigmas	2
1.2. Variables y tipos de datos	2
Tipos básicos en Python	3
1.3. Operadores	4
Operadores Aritméticos.....	4
Operadores de Comparación	4
Operadores Lógicos.....	4
1.4. Condicionales en Python.....	5
Condición simple	5
Condición doble.....	5
Condición múltiple	5
1.5. Bucles en Python	6
Bucle for	6
Bucle While	6
Instrucciones útiles en los bucles.....	7
1.6. Listas.....	7
1.7. Uso de ficheros	8
Leer de un fichero	9
Escribir en un fichero.....	9
1.8. Excepciones en Python.....	10
Excepción Base	10
Excepciones del Sistema	10
Excepciones Estándar Más Comunes	10
Ejemplos del uso de excepciones	11
1.9. POO	12

1. Fundamentos de Python: bases y orientación a objetos. Parte 1

Python es un **lenguaje de alto nivel** de programación interpretado cuya filosofía hace hincapié en la legibilidad de su código. Se trata de un lenguaje de programación multiparadigma, ya que soporta parcialmente la orientación a objetos, programación imperativa y, en menor medida, programación funcional. **Es un lenguaje interpretado, dinámico y multiplataforma.**

Administrado por *Python Software Foundation*, posee una licencia de código abierto, denominada *Python Software Foundation License*. Python se clasifica constantemente como uno de los lenguajes de programación más populares, siendo ya en 2025 el más popular y además con una amplia diferencia récord histórico de más de 15 puntos porcentuales sobre el siguiente.

1.1. Características y Paradigmas

Python es un lenguaje de programación multiparadigma. Esto significa que más que forzar a los programadores a adoptar un estilo particular de programación, permite varios estilos: programación orientada a objetos, programación imperativa y programación funcional. Otros paradigmas están soportados mediante el uso de extensiones.

Python usa **tipado dinámico** y **conteo de referencias** para la gestión de memoria.

```
x = 0
x = "Hola"

x = x + " a todos"
print(x)
```

Salida:

```
Hola a todos
```

1.2. Variables y tipos de datos

En Python no es necesario declarar el tipo de las variables explícitamente. Algunos tipos básicos son:

```
nombre = "Francisco"  
edad = 33  
altura = 1.79  
vivo = True
```

Aquí hemos creado 4 variables:

- Nombre -> tipo texto
- Edad -> entero
- Altura -> valor decimal
- Vivo -> valor lógico

Tipos básicos en Python

```
#int  
x = 42  
y = -7  
#float  
pi = 3.1416  
temperatura = -2.5  
#str  
saludo = "Hola, mundo"  
#bool  
estudiante = True  
aprobado = False
```

En Python **no hace falta indicar el tipo** de la variable, se detecta automáticamente según el valor que le asignemos.

Podemos **cambiar el tipo de dato** de una variable simplemente asignando otro valor.

¿Cómo podemos comprobar el tipo de una variable?

Con la función `type()` podemos comprobar del tipo que es una variable.

```
x = 3.14  
print(type(x))
```

Salida:

```
<class 'float'>
```

1.3. Operadores

Los **operadores** permiten realizar operaciones con variables y valores.

Operadores Aritméticos

Operador	Descripción	Ejemplo	Resultado
+	Suma	5 + 3	8
-	Resta	5 - 3	2
*	Multiplicación	5 * 3	15
/	División	5 / 2	2.5
//	División entera	5 // 2	2
%	Módulo	5 % 2	1
**	Potencia	2 ** 3	8

Operadores de Comparación

Operador	Descripción	Ejemplo	Resultado
==	Igual que	5 == 5	True
!=	Distinto que	5 != 3	True
>	Mayor que	5 > 3	True
<	Menor que	5 < 3	False
>=	Mayor o igual	5 >= 5	True
<=	Menor o igual	3 <= 5	True

Operadores Lógicos

Operador	Descripción	Ejemplo	Resultado
and	Devuelve True si ambas condiciones son verdaderas	(5 > 3 and 4 < 6)	True

or	Devuelve True si (5 > 3 or 4 > 6) al menos una condición es verdadera	True
not	Niega una not(5 > 3) condición	False

1.4. Condicionales en Python

Los **condicionales** permiten que un programa tome decisiones dependiendo de si una condición es **verdadera (True)** o **falsa (False)**.

Condicional simple

Ejecuta un bloque de código **solo si** la condición se cumple.

```
#Condicional simple
edad = 20

if edad >= 18:
    print("Eres mayor de edad")
```

Condicional doble

Si la condición se cumple, ejecuta un bloque; **si no**, ejecuta otro.

```
#Condicional doble
edad = 16

if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```

Condicional múltiple

Se pueden evaluar **varias condiciones** usando elif (else if).

```
#Condicional múltiple
nota = 7

if nota >= 9:
    print("Sobresaliente")
elif nota >= 7:
    print("Notable")
elif nota >= 5:
    print("Aprobado")
else:
    print("Suspenso")
```

1.5. Bucles en Python

Un **bucle** es una estructura que permite **repetir un bloque de instrucciones** varias veces.

En Python existen principalmente dos tipos de bucles: for y while.

Bucle for

Se utiliza para recorrer elementos de una secuencia (listas, cadenas, rangos de números, etc.).

```
#BUCLE FOR
ciclos = ["DAM1", "ASIR1", "DAF1"]
for ciclo in ciclos:
    print(ciclo)
```

También existe la posibilidad de utilizar in range para un número determinado de números.

```
for i in range(3):
    print(i)

for i in range(3,10):
    print(i)
```

Bucle While

Se repite **mientras** la condición sea verdadera (True)

```
contador = 0  
  
while contador < 5:  
    print(contador)  
    contador += 1
```

Instrucciones útiles en los bucles

Break -> interrumpe el bucle.

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

Continue -> salta a la siguiente iteración

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

Else -> se ejecuta cuando el bucle termina con normalidad.

```
for i in range(3):  
    print(i)  
else:  
    print("Bucle terminado")
```

1.6. Listas

Una lista es una estructura de datos que permite guardar múltiples valores en una sola variable. Pueden contener cualquier tipo de dato.

```

dias = []
#Append Añade un único elemento al final de la lista.
dias.append("Lunes")
dias.append("Martes")
dias.append("Miercoles")
print(dias)
#Remove Remueve la primer coincidencia del elemento especificado.
dias.remove("Martes")
print(dias)
print("CORTO")
elem = dias.pop()
print(elem)
print(dias)

print(dias[-1])
fines_semana = ['Sabado', 'Domingo']
#extends Añade otra lista al final de una lista.
dias.extend(fines_semana)
print("Ya esta la lista")
print(dias)

#del Elimina el elemento ubicado en el indice descrito
numeros = [1,2,3,4,5,6]
del numeros[1]
print(numeros)
del numeros[:2]
print(numeros)

#insert Inserta un nuevo elemento en una posición determinada de la lista
numeros.insert(_index: 0, _object: 10)
print(numeros)

#Sort para ordenar
numeros.sort()
print("Voy a imprimir ordenado")
print(numeros)
numeros.sort(reverse=True)

```

1.7. Uso de ficheros

Abrir un fichero

La función `open()` se utiliza para abrir un fichero en Python. Tiene dos parámetros importantes:

El nombre del fichero que quieres abrir.

El modo en que quieres abrir el fichero (lectura, escritura, etc.).

```
f = open("nombre_fichero.txt", "modo")
```

Los modos más comunes son:

- "r": Leer. El fichero debe existir.
- "w": Escribir. Si el fichero no existe, se crea; si existe, se sobrescribe.
- "a": Añadir. Escribe al final del fichero sin sobrescribir el contenido existente.
- "r+": Leer y escribir.

Leer de un fichero

```
f = open("archivo.txt", "r")
print(f.read())
#Después de trabajar con un fichero, es importante
# cerrarlo para liberar recursos. Usamos el método close() para ello:
f.close()
```

Otra forma de leer un fichero sería con el uso de with en el que al terminar el bloque de código se cerraría de forma automática la transacción.

```
with open("variasLineas.txt", "r") as f:
    linea = f.readline()
    while linea:
        print(linea, end="")
        linea = f.readline()
```

Escribir en un fichero

```
'''
write()
Escribe una cadena en el fichero. Si el fichero no existe, se crea. Si ya existe, se sobrescribe (si usas el modo "w")
'''
with open("nuevo_fichero.txt", "w") as f:
    f.write("Buenos días, a por el jueves.\n")

'''
writelines()
Escribe una lista de cadenas en el fichero.
'''
lineas = ["Primera línea\n", "Segunda línea\n", "Tercera línea\n"]

with open("nuevo_fichero.txt", "w") as f:
    f.writelines(lineas)

'''
Añadir a un fichero ("a")
Si deseas añadir contenido a un fichero sin sobrescribir el existente, usa el modo "a".
'''
with open("nuevo_fichero.txt", "a") as f:
    f.write("Añadiendo esta nueva línea al final.\n")
```

Tras la ejecución del código tenemos que comprobar que se han creado los ficheros en nuestro directorio raíz los ficheros correspondientes.

≡ nuevo_fichero.txt

```
Primera línea
Segunda línea
Tercera línea
Añadiendo esta nueva línea al final.
```

1.8. Excepciones en Python

Excepción Base

- **BaseException:** Es la clase base para todas las excepciones. No deberías manejarla directamente.
- **Exception:** Clase base para la mayoría de las excepciones. Se suele usar para capturar cualquier excepción que herede de ella.

Excepciones del Sistema

- **SystemExit:** Se lanza cuando se usa `sys.exit()` para salir del programa.
- **KeyboardInterrupt:** Se lanza cuando el usuario interrumpe la ejecución del programa (generalmente pulsando Ctrl+C).
- **GeneratorExit:** Se lanza cuando un generador o coroutine finaliza.

Excepciones Estándar Más Comunes

ArithmeticError

- **ZeroDivisionError:** División por cero.
- **OverflowError:** Un número es demasiado grande para ser representado.
- **FloatingPointError:** Error en operaciones de punto flotante.

LookupError

- **IndexError:** Cuando un índice está fuera del rango en una secuencia.
- **KeyError:** Cuando una clave no existe en un diccionario.

Otras comunes

- **ValueError:** Una función recibe un argumento del tipo correcto, pero con un valor inapropiado.
- **TypeError:** Una operación o función recibe un objeto de un tipo inapropiado.

- **AttributeError**: Se intenta acceder a un atributo que no existe en un objeto.
- **ImportError**: Una importación falla.
- **ModuleNotFoundError**: No se encuentra el módulo que se intenta importar.
- **NameError**: Una variable o nombre no está definido.
- **UnboundLocalError**: Una variable local se usa antes de ser asignada.
- **FileNotFoundError**: Archivo o directorio no encontrado.
- **IOError**: Error en una operación de entrada/salida. En Python 3.x está unificada con **OSError**.
- **OSError**: Error relacionado con el sistema operativo (archivos o procesos).
- **PermissionError**: Se intenta realizar una operación sin los permisos adecuados.
- **FileExistsError**: Se intenta crear un archivo que ya existe.
- **IsADirectoryError**: Se trata un directorio como si fuera un archivo.
- **RuntimeError**: Error que no pertenece a ninguna otra categoría.
- **RecursionError**: Recursión demasiado profunda.
- **MemoryError**: El programa no puede asignar memoria adicional.
- **StopIteration**: Un iterador no tiene más elementos.
- **StopAsyncIteration**: Un objeto asíncrono no tiene más elementos.
- **AssertionError**: Falla una declaración `assert`.

Ejemplos del uso de excepciones

```
def divisionZero():
    try:
        resultado = 10 / 0
    except ZeroDivisionError:
        print("No se puede dividir entre cero.")

def multiple():
    try:
        numero = int(input("Introduce un número: "))
        resultado = 10 / numero
    except ValueError:
        print("Debes introducir un número válido.")
    except ZeroDivisionError:
        print("No se puede dividir entre cero.")
```

```

def conFinally():
    try:
        archivo = open("archivo.txt", "r")
        contenido = archivo.read()
    except FileNotFoundError:
        print("El archivo no se encontró.")
    finally:
        archivo.close()

1 usage
def verificar_numero(numero):
    if numero < 0:
        raise ValueError("El número no puede ser negativo.")
    return numero

def lanzarExcepciones():
    try:
        verificar_numero(-5)
    except ValueError as e:
        print(f"Error: {e}")

```

1.9. POO

<https://github.com/alion1992/PAUF>